

rules_project

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

RulesProject

```
RulesProject(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a Rules Project in Pulse.

actions property writable

```
actions: dict[str, list[WorkflowActionParam]]
```

Get or set the actions that the rules project can take.

app property

```
app
```

Get the rules project's application

desc property writable

```
desc: str
```

Get or set the user visible name of this rules project.

evaluations property

```
evaluations: dict[str, RulesProjectEvaluation]
```

Get a mapping of this project's snapshots and their evaluations.

The key of the dictionary is the name of the snapshot, and the value is the corresponding snapshot evaluation object.

If a snapshot hasn't been evaluated, no entry for that snapshot is included in the dictionary.

id property

```
id: str
```

Get the rules project's id.

rules property

```
rules: list[Rule]
```

Get the rules belonging to this rules project.

schema property [¶](#)

```
schema: Schema
```

Get the schema associated with this rules project.

snapshots property [¶](#)

```
snapshots: dict[str, RulesProjectSnapshot]
```

Get a mapping of this project's snapshot objects and their names.

The key of the dictionary is the name of the snapshot, and the value is the corresponding snapshot object.

subclasses class-attribute instance-attribute [¶](#)

```
subclasses: List[JavaWrapper] = []
```

target_field property writable [¶](#)

```
target_field: str
```

Get or set the target field of this rules project.

tenancy_type property [¶](#)

```
tenancy_type: RulesProjectTenancyTypeEnum
```

Get the tenancy type of this rules project.

create classmethod [¶](#)

```
create(app: App, name: str, schema: Schema, target_field: str, actions: Mapping[str, Sequence[WorkflowActionParam]] | None = None) -> RulesProject
```

Creates and returns a new Rules Project.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	App	Application ds_api object.	<i>required</i>
<code>name</code> ¶	str	Name of the new rules project.	<i>required</i>
<code>schema</code> ¶	Schema	Pulse schema	<i>required</i>
<code>target_field</code> ¶	str	Target field for the rules decision.	<i>required</i>
<code>actions</code> ¶	Mapping[str, Sequence[WorkflowActionParam]]	Mapping between the possible actions for the rules to trigger, and the respective parameters. Default is None.	None

Returns:

Type	Description
RulesProject	

Type	Description
	New RulesProject.

create_rule [¶](#)

```
create_rule(name: str, condition: str, variables: str | None = None, comment: str | None = None, actions: Set[str] | None = None, is_enabled: bool = True, control_flow: ControlFlowActionTypeEnum | None = None, managed_lists_id: Set[str] | None = None, reload: bool = True) -> Rule
```

Create a new rule within this rules project.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the rule to add.	<i>required</i>
<code>condition</code> ¶	<code>str</code>	The expression of the rule's triggering condition.	<i>required</i>
<code>variables</code> ¶	<code>str</code>	The rule's variables field, by default None.	None
<code>comment</code> ¶	<code>str</code>	The comment describing this rule, by default None.	None
<code>actions</code> ¶	<code>Set[str]</code>	Which actions are triggered by this rule, by default None.	None
<code>is_enabled</code> ¶	<code>bool</code>	Boolean identifying whether the rule is enabled or not, by default True.	True
<code>control_flow</code> ¶	<code>ControlFlowActionTypeEnum</code>	Control flow action to take after evaluating a rule, by default <code>ControlFlowActionTypeEnum.CONTINUE</code> .	None
<code>managed_lists_id</code> ¶	<code>Set[str]</code>	The lists associated with the rule, by default None.	None
<code>reload</code> ¶	<code>bool</code>	If the rules project should be reloaded to reflect the added rule, by default True. This could be used when creating multiple rules, to avoid reloading on every creation.	True

Returns:

Type	Description
<code>Rule</code>	The newly created rule.

create_snapshot [¶](#)

```
create_snapshot(snapshot_desc: str | None = None, include_rules: Sequence[str | RulesProjectSnapshot] | None = None, find_by: str | None = None, reload: bool = True) -> RulesProjectSnapshot
```

Create a snapshot of the rules project and some/all of its rules.

Parameters:

Name	Type	Description	Default
snapshot_desc ¶	str	Name given to the snapshot. If None is provided, a snapshot name is given using the format: %m-%d-%Y_%H-%M-%S_project-desc .	None
include_rules ¶	Sequence[str Rule]	The list of rule objects, rule ids or rule names which uniquely identify the rules. If None, then all rules in the project are included in the snapshot, which is the default behavior.	None
find_by ¶	('object', 'id', 'desc')	Name of the property of rules (or the rule objects themselves), which describes the values given to by the <code>include_rules</code> parameter: • 'object' : Default behaviour. Include the given rule objects. • 'id' : Include the rules in the project, given the list of rule ids. • 'desc' : Include the rules in the project, following the order of the given rule's user visible names.	'object'
reload ¶	bool	If this rules project should be reloaded, by default True.	True

Returns:

Type	Description
RulesProjectSnapshot	The newly created rules project snapshot.

delete_rule [¶](#)

```
delete_rule(rule: str, find_by: str, reload: bool = True) -> None
```

Delete a rule associated with this rules project.

Parameters:

Name	Type	Description	Default
rule ¶	str	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to delete.	<i>required</i>
find_by ¶	('id', 'desc')	Name of the property of a Rule, which uniquely identifies the rule to delete, which describes the given <code>rule</code> : • 'id' : delete rule with the given id. • 'desc' : delete rule with the given user visible name.	'id'
reload ¶	bool	If the rules project should be reloaded to reflect the deleted rule, by default True.	True

delete_snapshot [↗](#)

```
delete_snapshot(snapshot: str, find_by: str, reload: bool = True) -> None
```

Delete a snapshot associated with this rules project.

Parameters:

Name	Type	Description	Default
snapshot ↗	str	The value of the snapshot property, specified by <code>find_by</code> , which identifies the snapshot to delete.	<i>required</i>
find_by ↗	('id', 'desc')	Name of the property of a snapshot, which uniquely identifies the snapshot to delete, which describes the given <code>snapshot</code> : 'id' : delete snapshot with the given id. 'desc' : delete snapshot with the given user visible name.	'id'
reload ↗	bool	If the rules project should be reloaded to reflect the deleted snapshot, by default True.	True

duplicate_rule [↗](#)

```
duplicate_rule(rule: str | Rule, find_by: str = OBJECT, new_desc: str | None = None, new_control_flow: ControlFlowActionTypeEnum | None = None, new_comment: str | None = None, new_actions: Set[str] | None = None, new_variables: str | None = None, new_condition: str | None = None, reload: bool = True) -> Rule
```

Duplicate the given rule within this project.

Optionally, you can change some attributes of the duplicate.

If an object is provided to the `rule` parameter, the rule doesn't have to belong to this project. It will take the properties of the given rule, and create a new one within this project with those properties.

Parameters:

Name	Type	Description	Default
rule ↗	str Rule	The value of the rule property, or the rule object itself, specified by <code>find_by</code> , which identifies the rule to duplicate.	<i>required</i>
find_by ↗	('id', 'desc', 'object')	Name of the property of a Rule, which uniquely identifies the rule to update (or the rule object itself), which describes the given <code>rule</code> parameter: 'id' : duplicate rule with the given id. 'desc' : duplicate rule with the given user visible name. (Default) 'object' : duplicate the given rule.	'id'
new_desc ↗	str	The name of the duplicated rule. Defaults to the original name suffixed with '_duplicate'.	None
new_control_flow ↗	ControlFlowActionTypeEnum None	The control flow to use in the duplicate. Defaults to the original.	None

Name	Type	Description	Default
<code>new_actions</code> ¶	<code>Set[str] None</code>	The actions to used in the duplicate. Defaults to the original.	<code>None</code>
<code>new_comment</code> ¶	<code>str</code>	The comment to use in the duplicate. Defaults to the original.	<code>None</code>
<code>new_variables</code> ¶	<code>str</code>	The variables to use in the duplicate. Defaults to the original.	<code>None</code>
<code>new_condition</code> ¶	<code>str</code>	The condition to use in the duplicate. Defaults to the original.	<code>None</code>
<code>reload</code> ¶	<code>bool</code>	If the rules project should be reloaded to reflect the added rule, by default True. This could be used when creating multiple rules, to avoid reloading on every creation. By default True.	<code>True</code>

Returns:

Type	Description
<code>Rule</code> ¶	The duplicated rule.

Raises:

Type	Description
<code>ValueError</code>	Raised if a rule with the same name exists within the rules project.

`evaluate` [¶](#)

```
evaluate(input_datasource: DataSource, evaluation_config: RulesProjectEvaluationConfig, snapshot_desc: str | None = None, include_rules: list[str] | None = None, find_by: str | None = None, profile_file_name: str | None = None, spark_job_properties: Mapping[str, str] | None = None, wait: bool = True, reload: bool = True) -> RulesProjectEvaluation
```

Create a snapshot and evaluate it on the given input data source.

Both spark job profile name and profile properties can be passed at the same time, but the profile name is prioritized.

Parameters:

Name	Type	Description	Default
<code>input_data_source</code> ¶	<code>DataSource</code> ¶	Data to run through the rules project to calculate alerts.	<i>required</i>
<code>evaluation_config</code> ¶	<code>RulesProjectEvaluationConfig</code> ¶	Configuration for the snapshot evaluation.	<i>required</i>
	<code>str</code>	Name given to the snapshot. If none is provided, a snapshot name is given using the format: <code>%m-%d-%Y_%H-%M-%S_project-desc</code> .	<code>None</code>

Name	Type	Description	Default
<code>snapshot_desc</code> ¶			
<code>include_rules</code> ¶	Sequence[str RulesProjectSnapshot]	The list of snapshots objects, snapshots ids or snapshots names which uniquely identify the rules. If None, then all rules in the project are included in the snapshot, which is the default behavior.	None
<code>find_by</code> ¶	('object', 'id', 'desc')	Name of the property of rules (or the rule objects themselves), which describes the values given to by the <code>include_rules</code> parameter: • 'object' : Default behaviour. Include the given rule objects. • 'id' : Include the rules in the project, given the list of rule ids. • 'desc' : Include the rules in the project, following the order of the given rule's user visible names.	'object'
<code>profile_name</code> ¶	str None	Job profile name, by default None	None
<code>spark_job_properties</code> ¶	Mapping None	Job profile properties, by default None	None
<code>wait</code> ¶	bool	Waits for the snapshot evaluation to finish before returning the evaluation object.	True
<code>reload</code> ¶	bool	If the rules project should be reloaded to reflect the created snapshot and evaluation, by default True.	True

Returns:

Type	Description
<code>RulesProjectEvaluation</code> ¶	The evaluation object containing its status.

from_dict [¶](#) classmethod [¶](#)

```
from_dict(app: App, rules_project_as_dict: dict[str, Any]) -> RulesProject
```

Create a Rules Project, given its dictionary representation.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code> ¶	The app in which to create the rules project on.	<i>required</i>
<code>rules_project_as_dict</code> ¶	dict[str, Any]	The dictionary representation of the rules project. It must have the following keys: desc : str User visible name of the rules project to create schema : str Name of the schema to be used for the project, which must already exist in the app target_field : str Target field for the rules decision. actions : dict[str, dict[str, Any], optional Mapping between the action names, and the action parameters. The available parameters are:	<i>required</i>

Name	Type	Description	Default
		<pre> param_desc : str, The user visible name of the action parameter. param_type : str The name of the parameter type, which must be the name of an ActionParamTypeEnum member. param_is_mandatory : bool A flag defining if the parameter is mandatory or not. param_resource_id : str The Id of the resource used if the type requires that information (e.g.: ActionParamTypeEnum.MANAGED_LIST). Default is None rules: Sequence[dict[str, Any]], optional The rules to be created, represented as a dictionary, following the same representation expected from Rule.from_dict </pre>	

Returns:

Type	Description
RulesProject	The Rules Project created from the given dictionary.

[get_rule](#)

```
get_rule(rule: str, find_by: str) -> Rule | None
```

Retrieve a rule associated with this Rules Project.

Parameters:

Name	Type	Description	Default
rule 	str	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to get.	<i>required</i>
find_by 	('id', 'desc')	Name of the property of a Rule, which uniquely identifies the rule to get, which describes the given <code>rule</code> : 'id' : get rule with the given id. 'desc' : get rule with the given user visible name.	'id'

Returns:

Type	Description
Rule None	The Rule with the provided <code>find_by</code> property or None if it doesn't exist.

[get_snapshot](#)

```
get_snapshot(snapshot: str, find_by: str) -> RulesProjectSnapshot
```

Retrieve a rule associated with this Rules Project.

Parameters:

Name	Type	Description	Default
	str		<i>required</i>

Name	Type	Description	Default
snapshot ¶		The value of the snapshot property, specified by <code>find_by</code> , which identifies the snapshot to get.	
find_by ¶	<code>('id', 'desc')</code>	Name of the property of a snapshot, which uniquely identifies the snapshot to get, which describes the given <code>snapshot</code> : 'id' : get snapshot with the given id. 'desc' : get snapshot with the given user visible name.	<code>'id'</code>

Returns:

Type	Description
RulesProjectSnapshot <code>None</code>	The Rules Project Snapshot with the provided <code>find_by</code> property or <code>None</code> if it doesn't exist.

reorder_rules [¶](#)

```
reorder_rules(rules_in_order: Sequence[Union[str, Rule]], find_by: str = OBJECT, tenant_id: str | None = None) -> None
```

Update the order of this project's rules in the context of a tenant.

Parameters:

Name	Type	Description	Default
rules_in_order ¶	<code>Sequence[Union[str, Rule]]</code>	The sorted list of rule objects, rule ids or rule names which uniquely identify the rules.	<i>required</i>
find_by ¶	<code>('object', 'id', 'desc')</code>	Name of the property of rules (or the rule objects themselves), which describes the values given to by the <code>rules_in_order</code> parameter: 'object' : Default behaviour. Re-order the rules in the project, following the order of the given rules instances. 'id' : Re-order the rules in the project, following the order of the given rule ids. 'desc' : Re-order the rules in the project, following the order of the given rule's user visible names.	<code>'object'</code>
tenant_id ¶	<code>str</code>	The id of the tenant where this operation is being performed, default is <code>None</code> .	<code>None</code>

Returns:

Type	Description
<code>bool</code>	True if any of the rules has been updated, false otherwise.

shift_rule [¶](#)

```
shift_rule(rule: str, find_by: str, offset: int) -> None
```

Moves the rule the number of positions specified by the offset.

This will in shift this rule in the rules project's ordered list of rules, copying the rule order value between affected rules. Therefore, even if the order values are not continuous (e.g. 1, 3, 5) they will be preserved as a whole but assigned to different rules (i.e. after any shift the order values will still be 1, 3 and 5).

A rule will only be shifted forward (positive offset) or backwards (negative offset) at most until it reaches the end or beginning of the list, respectively - i.e. the list is not treated like a circular buffer.

Parameters:

Name	Type	Description	Default
<code>rule</code> ¶	<code>str</code>	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to shift.	<i>required</i>
<code>find_by</code> ¶	<code>('id', 'desc')</code>	Name of the property of rules, which uniquely identifies the rule to shift, whose value is provided by the <code>rule</code> : • 'id' : shift rule with the given id. • 'desc' : shift rule with the given user visible name.	<code>'id'</code>
<code>offset</code> ¶	<code>int</code>	The number of positions to move the rule. It can be positive to move the rule <code>offset</code> positions forward or negative to move it backwards.	<i>required</i>

`to_dict` [¶](#)

```
to_dict(include_params: Sequence[str] | None = None) -> dict[str, Any]
```

Get a dictionary representation of this project and its rules.

Parameters:

Name	Type	Description	Default
<code>include_params</code> ¶	<code>Sequence[str]</code>	Properties of the rules project to include in the dictionary. Can be one of {'id', 'desc', 'app_id', 'tenancy_type', 'schema_id', 'target_field', 'actions', 'rules'}	<code>None</code>

Returns:

Type	Description
<code>dict[str, Any]</code>	A DataFrame with the properties for all rules in the project, along with the project's properties.

`update_rule` [¶](#)

```
update_rule(rule: str, find_by: str, reload: bool = True, **kwargs: Mapping[str, Any]) -> None
```

Update several properties of a rule associated with this project.

Warning

This method doesn't check whether the properties you pass as a kwarg is one of the existing modifiable ones, so it's up to the caller to make sure the keys are correct.

Parameters:

Name	Type	Description	Default
<code>rule</code> ¶	<code>str</code>	The value of the rule property, specified by <code>find_by</code> , which identifies the rule to update.	<i>required</i>

Name	Type	Description	Default
<code>find_by</code> ¶	<code>('id', 'desc')</code>	Name of the property of a Rule, which uniquely identifies the rule to update, which describes the given rule : 'id' : update rule with the given id. 'desc' : update rule with the given user visible name.	<code>'id'</code>
<code>reload</code> ¶	<code>bool</code>	If the rules project should be reloaded to reflect the added rule, by default True. This could be used when creating multiple rules, to avoid reloading on every creation. By default True.	<code>True</code>
<code>**kwargs</code> ¶	<code>Mapping[str, Any]</code>	Map between rule property names and the values we want to update them to	<code>{}</code>

Raises:

Type	Description
<code>TypeError</code>	If any of the given property values aren't of the required type

RulesProjectTenancyTypeEnum [¶](#)

Bases: [JavaEnumWrapper](#)

Possible types of tenancy for rules projects.

MULTI_TENANT_DISTINCT `class-attribute` `instance-attribute` [¶](#)

```
MULTI_TENANT_DISTINCT = auto()
```

Multi-tenant distinct. Each tenant may have its individual rules.

MULTI_TENANT_SHARED `class-attribute` `instance-attribute` [¶](#)

```
MULTI_TENANT_SHARED = auto()
```

Multi-tenant shared. Rules are common to all tenants but are processed independently by each tenant. This means events are filtered by tenant on evaluations, and that on stateful rules each tenant has a dedicated rules engine.

SINGLE_TENANT `class-attribute` `instance-attribute` [¶](#)

```
SINGLE_TENANT = auto()
```

Single tenant, the default. There is no tenant segregation.